Conductivity Pipeline

Introduction

This vignette walks through the complete conductivity processing pipeline using the `waterlogged` package. We use real test data from the TUMT_WE_40 wetland station as our example.

The pipeline processes raw HOBO U24 conductivity logger data through these steps:

1. Read raw conductivity logger files and link them to metadata
2. Run automatic quality checks (spikes, ice, dry sensor)
3. Review plots to identify problem periods
4. Correct sensor drift using field reference measurements
5. Convert electrical conductivity to specific conductance (SPC)
6. Make manual corrections as needed
7. Export the final clean dataset

What is conductivity? Electrical conductivity (EC) measures how well water conducts electricity, reported in microsiemens per centimetre (µS/cm). Higher conductivity means more dissolved ions (salts, minerals). It's a quick indicator of water chemistry changes — runoff events, groundwater inputs, or pollution can all shift conductivity.

Why temperature compensation? Conductivity varies with water temperature — warmer water conducts better. To compare readings across seasons (or between sites at different temperatures), we normalize to a standard temperature (25 °C). This normalized value is called "specific conductance" (SPC) and is the standard for reporting.

No barometric data needed: Unlike water level and DO, conductivity does not require atmospheric pressure correction. This makes the pipeline shorter.

This vignette assumes you have already installed `waterlogged` from the develop branch:

```
install.packages("pak")
pak::pkg_install("dfo-pacific-science/waterlogged@develop")
```

Setup

```
library(waterlogged)
```

Define paths to the test data shipped with the package.

```
metadata_file  <- system.file("extdata/conductivity/metadata.csv", package = "waterlogged")
cond_folder    <- system.file("extdata/conductivity/COND", package = "waterlogged")
reference_file <- system.file("extdata/conductivity/field_references.csv", package = "waterlogged")
```

Step 1: Bind Raw Conductivity Logger Files (v0.1)

Reads the raw HOBO U24 conductivity logger CSV and links it to the deployment metadata.

The HOBO U24 records two (or three) channels:

- Low Range conductivity (µS/cm) — used for most freshwater
- Full Range conductivity (µS/cm) — for higher conductivity water
- Water temperature (°C)

This step parses them into `conduct_uScm` and `watertemp_C`. When both Low and Full Range are present, the function selects the appropriate one and records the source in `conduct_source`.

About the TUMT_WE_40 data:

- Logger: HOBO U24 (SN 21245027)
- Deployed: April 17 to May 28, 2025 (~6 weeks)
- Location: TUMT wetland, station WE_40
- One download file covering the full deployment

```
cond_bound <- bind_hobo_files(
  path_to_raw_folder = cond_folder,
  metadata_path      = metadata_file
)

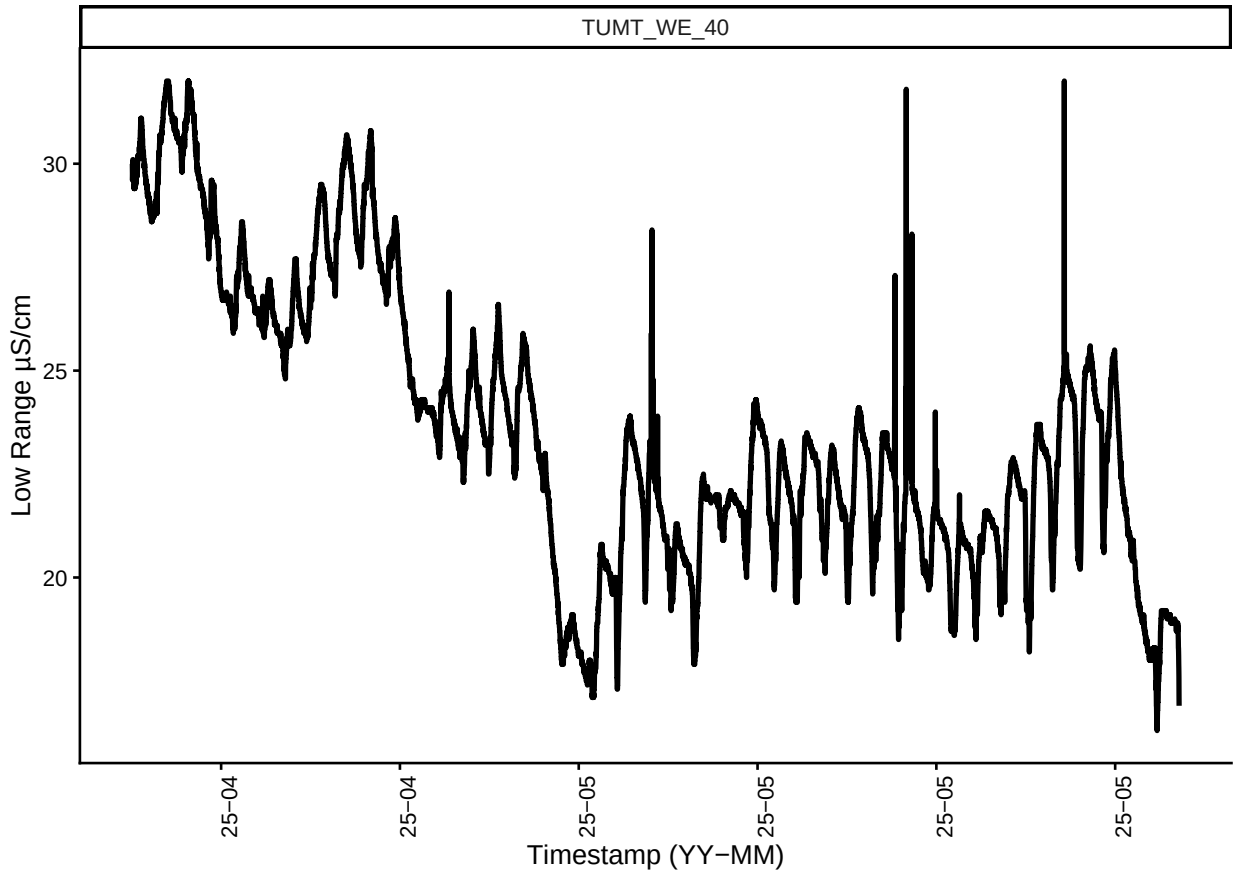
cond_data <- cond_bound[[1]]
cond_plot <- cond_bound[[2]]

cat("Conductivity data:", nrow(cond_data), "rows\n")
#> Conductivity data: 11804 rows
cat("Stations:", paste(unique(cond_data$site_station_code), collapse = ", "), "\n")
#> Stations: TUMT_WE_40
cat("Date range:", as.character(min(cond_data$timestamp)),
    "to", as.character(max(cond_data$timestamp)), "\n")
#> Date range: 2025-04-17 12:30:00 to 2025-05-28 12:05:00
cat("Conductivity range:",
    round(min(cond_data$conduct_uScm, na.rm = TRUE), 1), "to",
    round(max(cond_data$conduct_uScm, na.rm = TRUE), 1), "uS/cm\n")
#> Conductivity range: 16.3 to 32 uS/cm
cat("Water temp range:",
    round(min(cond_data$watertemp_C, na.rm = TRUE), 1), "to",
    round(max(cond_data$watertemp_C, na.rm = TRUE), 1), "deg C\n")
#> Water temp range: 5.5 to 15.6 deg C
```

```
if ("conduct_source" %in% names(cond_data)) {
  cat("Conductivity source channels used:\n")
  print(table(cond_data$conduct_source))
}
#> Conductivity source channels used:
#>
#>   low
#> 11804
```

```r
head(cond_data[, c("timestamp", "conduct_uScm", "watertemp_C",
                   "site_station_code")])
#> # A tibble: 6 x 4
#>   timestamp           conduct_uScm watertemp_C site_station_code
#>   <dttm>                     <dbl>       <dbl> <chr>
#> 1 2025-04-17 12:30:00         30          7.12 TUMT_WE_40
#> 2 2025-04-17 12:35:00         30          7.22 TUMT_WE_40
#> 3 2025-04-17 12:40:00         29.8        7.09 TUMT_WE_40
#> 4 2025-04-17 12:45:00         29.6        6.99 TUMT_WE_40
#> 5 2025-04-17 12:50:00         29.7        7.12 TUMT_WE_40
#> 6 2025-04-17 12:55:00         29.8        7.3  TUMT_WE_40
```

```r
cond_plot
```



Step 2: Automatic Conductivity QAQC

Runs automatic quality checks on the conductivity data and flags suspicious values.

Conductivity sensors can be affected by ice, air exposure (dry), biofouling, or physical disturbances. This step catches common problems automatically.

What it checks:

1. **Spikes** — sudden large jumps in conductivity between consecutive readings (> 200 µS/cm)

2. **Ice** — water temperature below 0.1 °C (sensor may be frozen)
3. **Temp range** — temperature outside plausible limits (−2 to 40 °C)
4. **Dry sensor** — very low conductivity (< 10 µS/cm) or air-water temp difference < 2 °C
5. **Near-zero temp** — water temp between −1 and 0 °C (corrected to 0)

Adjusted columns created: `conduct_uScm_adj`, `watertemp_C_adj`.

Flag columns: `flag_co_spike`, `flag_co_temp_ice`, `flag_co_temp_low`, `flag_co_temp_high`, `flag_co_temp_neg_near`, `flag_co_dry`.

```
cond_qc <- conductivity_qaqc(
  input_data                 = cond_data,
  select_station             = "TUMT_WE_40",
  temp_low_limit_C           = -2,
  temp_high_limit_C          = 40,
  disturbance_threshold_uScm = 200,
  dry_threshold_uScm         = 10,
  air_water_diff_threshold_C = 2
)

cat("QAQC results for TUMT_WE_40:\n")
#> QAQC results for TUMT_WE_40:
cat("  Total rows:       ", nrow(cond_qc), "\n")
#>   Total rows:        11804
cat("  Spikes:           ", sum(cond_qc$flag_co_spike, na.rm = TRUE), "\n")
#>   Spikes:            0
cat("  Ice flagged:      ", sum(cond_qc$flag_co_temp_ice, na.rm = TRUE), "\n")
#>   Ice flagged:       0
cat("  Temp too low:     ", sum(cond_qc$flag_co_temp_low, na.rm = TRUE), "\n")
#>   Temp too low:      0
cat("  Temp too high:    ", sum(cond_qc$flag_co_temp_high, na.rm = TRUE), "\n")
#>   Temp too high:     0
cat("  Near-zero temp:   ", sum(cond_qc$flag_co_temp_neg_near, na.rm = TRUE), "\n")
#>   Near-zero temp:    0
cat("  Dry flagged:      ", sum(cond_qc$flag_co_dry, na.rm = TRUE), "\n")
#>   Dry flagged:       0
```

```
qaqc_log <- attr(cond_qc, "qaqc_log")
if (!is.null(qaqc_log) && nrow(qaqc_log) > 0) {
  cat("QAQC log entries:\n")
  print(qaqc_log[, c("field", "action", "code", "ts_start", "ts_end")])
}
#> QAQC log entries:
#> # A tibble: 1 x 5
#>   field            action    code  ts_start ts_end
#>   <chr>            <chr>     <chr> <chr>    <chr>
#> 1 conduct_uScm_adj AUTO_QAQC <NA>  <NA>     <NA>
```

Step 3: Visual Review

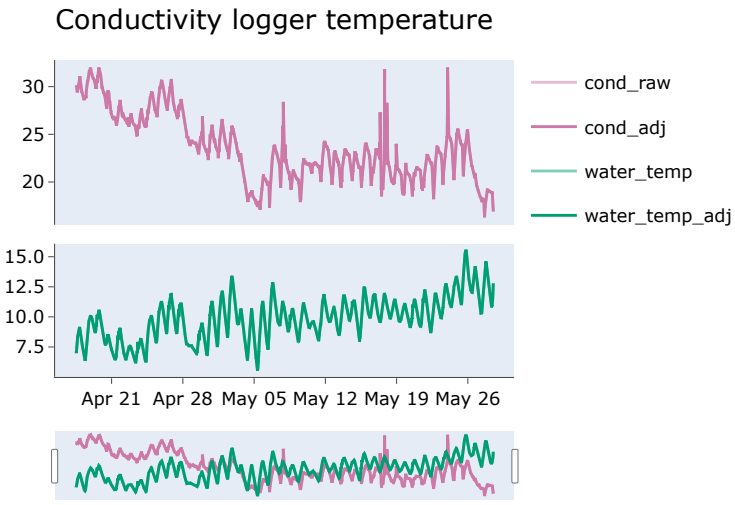
Creates an interactive plot for reviewing the conductivity data.

How to read the plot:

- Light traces = raw values (original, unmodified)
- Dark traces = adjusted values (after QAQC)
- Colored dots = flag locations
- Use the range slider to zoom into specific time periods

Look for: sudden jumps (disturbance, debris on sensor), flat low readings (sensor out of water), and gradual drift from biofouling.

```
plot_qaqc_timeseries(
  cond_data      = cond_qc,
  select_station = "TUMT_WE_40"
)
```


Conductivity logger temperature
30
25
20
15.0
12.5
10.0
7.5
Apr 21
Apr 28
May 05
May 12
May 19
May 26
cond_raw
cond_adj
water_temp
water_temp_adj

Step 4: Drift Correction

Adjusts the logged conductivity to match field reference measurements, correcting for sensor drift between site visits.

Conductivity sensors can drift due to biofouling (algae/biofilm coating the electrodes), mineral deposits, or calibration drift. Field measurements taken with a calibrated handheld meter give us "truth" values to correct against.

Our TUMT_WE_40 reference data:

- 4/21/2025: check (26 µS/cm) — initial check, no sensor touching
- 5/7/2025: calibration (25 µS/cm) — sensor cleaned and recalibrated
- 5/16/2025: calibration (28 µS/cm) — second recalibration
- 5/26/2025: check (27 µS/cm) — final check visit

Between each pair of reference events, the function calculates how far the logger drifted from truth and applies a linear correction:

- **"ratio" method** (default): Multiplicative correction. `correction_factor = reference_value / logger_value`, interpolated linearly from 1.0 at segment start.
- **"offset" method**: Additive correction. `correction_offset = reference_value - logger_value`, interpolated linearly from 0 at segment start.

```
ref_data <- read.csv(reference_file)
cat("Field reference measurements for TUMT_WE_40:\n")
#> Field reference measurements for TUMT_WE_40:
ref_data[ref_data$metric == "cond", ]
#>   metric site_station_code       timestamp watertemp_C value unit  event_type
#> 1   cond        TUMT_WE_40 4/21/2025 11:40         9.5    26 uScm       check
#> 2   cond        TUMT_WE_40   5/7/2025 9:05        17.6    25 uScm calibration
#> 3   cond        TUMT_WE_40 5/16/2025 12:42        12.2    28 uScm calibration
#> 4   cond        TUMT_WE_40 5/26/2025 15:00        10.0    27 uScm       check
#>   calibration_datetime comments
#> 1                           NA
#> 2        5/7/2025 9:10       NA
#> 3      5/16/2025 12:42       NA
#> 4                           NA
```

```
cond_qc <- wq_drift_correction(
  input_data     = cond_qc,
  ref_data_path  = reference_file,
  select_station = "TUMT_WE_40",
  metric         = "cond",
  method         = "ratio",
  max_ref_gap_h  = 1
)

if ("edit_cond_drift" %in% names(cond_qc)) {
  drift_count <- sum(cond_qc$edit_cond_drift, na.rm = TRUE)
  cat("Rows corrected by drift:", drift_count, "of", nrow(cond_qc), "\n")
} else {
  cat("No drift correction column found - check reference data alignment.\n")
}
#> Rows corrected by drift: 11260 of 11804
```

```
drift_log <- attr(cond_qc, "qaqc_log")
if (!is.null(drift_log) && nrow(drift_log) > 0) {
  cat("Drift correction log:\n")
  print(drift_log)
}
#> Drift correction log:
#> # A tibble: 1 x 13
#>   station  metric field action code  action_note manual_note ts_start
#>   <chr>    <chr>  <chr> <chr>  <chr> <chr>       <chr>       <dttm>
#> 1 TUMT_WE~ COND   cond~ DRIFT~ DRIF~ Forward li~ AUTOMATIC ~ 2025-04-17 12:35:00
#> # i 5 more variables: ts_end <dttm>, duration_hours <dbl>, fun_name <chr>,
#> #   run_at <dttm>, user <chr>
```

Step 5: Temperature Compensation (Specific Conductance)

Converts electrical conductivity (EC) to specific conductance (SPC) normalized to 25 °C.

Conductivity changes with water temperature — warmer water conducts electricity better. A reading of 100 µS/cm at 10 °C represents a different amount of dissolved ions than 100 µS/cm at 25 °C. Specific conductance removes this temperature effect, making it possible to:

- Compare readings across seasons (cold winter vs warm summer)
- Compare readings between sites at different temperatures
- Detect real water chemistry changes vs temperature changes

The math follows ISO 27888, using a 5th-order polynomial to model the temperature dependence of conductivity. The function calculates what the conductivity would be if the water were at exactly 25 °C.

Important: Run this after drift correction, so the temperature compensation uses the drift-corrected conductivity values.

```
cond_qc <- conductivity_temp_compensation(cond_qc)

cat("Temperature compensation complete.\n")
#> Temperature compensation complete.
cat("Raw conductivity range:   ",
    round(min(cond_qc$conduct_uScm_adj, na.rm = TRUE), 1), "to",
    round(max(cond_qc$conduct_uScm_adj, na.rm = TRUE), 1), "uS/cm\n")
#> Raw conductivity range:    16.3 to 39.5 uS/cm
cat("Specific conductance range:",
    round(min(cond_qc$spc_uScm_adj, na.rm = TRUE), 1), "to",
    round(max(cond_qc$spc_uScm_adj, na.rm = TRUE), 1), "uS/cm @ 25C\n")
#> Specific conductance range: 21.2 to 53.3 uS/cm @ 25C
```

```
cat("Sample comparison (EC vs SPC at varying temperatures):\n")
#> Sample comparison (EC vs SPC at varying temperatures):
sample_rows <- cond_qc[as.integer(seq(1, nrow(cond_qc), length.out = 6)),
                       c("timestamp", "watertemp_C_adj",
                         "conduct_uScm_adj", "spc_uScm_adj")]
print(sample_rows)
#> # A tibble: 6 x 4
#>   timestamp           watertemp_C_adj conduct_uScm_adj spc_uScm_adj
```

```
#>   <dttm>                <dbl>          <dbl>       <dbl>
#> 1 2025-04-17 12:30:00    7.12           30          46.4
#> 2 2025-04-25 17:10:00   10.7            30.4        42.6
#> 3 2025-05-03 21:55:00    9.42           23.0        33.4
#> 4 2025-05-12 02:35:00   10.6            28.5        40.0
#> 5 2025-05-20 07:20:00    9.14           23.2        33.8
#> 6 2025-05-28 12:05:00   12.8            16.9        22.4
```

Step 6: Manual Adjustments (Examples)

This section demonstrates how to manually remove bad data. Apply these only after reviewing the plot and identifying specific problem periods.

For conductivity data, the three reasons for removal are:

- **"ice"** — sensor frozen; conductivity unreliable. Sets `conduct_uScm_adj` and `watertemp_C_adj` to `NA`.
- **"dry"** — sensor out of water; readings are meaningless. Sets `conduct_uScm_adj` and `watertemp_C_adj` to `NA`.
- **"disturbance"** — physical disturbance. Sets `conduct_uScm_adj` and `watertemp_C_adj` to `NA`.

Remove a dry period

```
cond_qc <- adjust_logger_NA(
  input_data       = cond_qc,
  select_station   = "TUMT_WE_40",
  metric           = "conductivity",
  reason_to_adjust = "dry",
  timestamp_start  = "2025-05-20 00:00:00",
  timestamp_end    = "2025-05-22 00:00:00",
  manual_note      = "Sensor exposed during low water"
)
```

Remove a disturbance

```
cond_qc <- adjust_logger_NA(
  input_data       = cond_qc,
  select_station   = "TUMT_WE_40",
  metric           = "conductivity",
  reason_to_adjust = "disturbance",
  timestamp_start  = "2025-05-10 12:00:00",
  timestamp_end    = "2025-05-10 14:00:00",
  manual_note      = "Debris caught on sensor during storm"
)
```

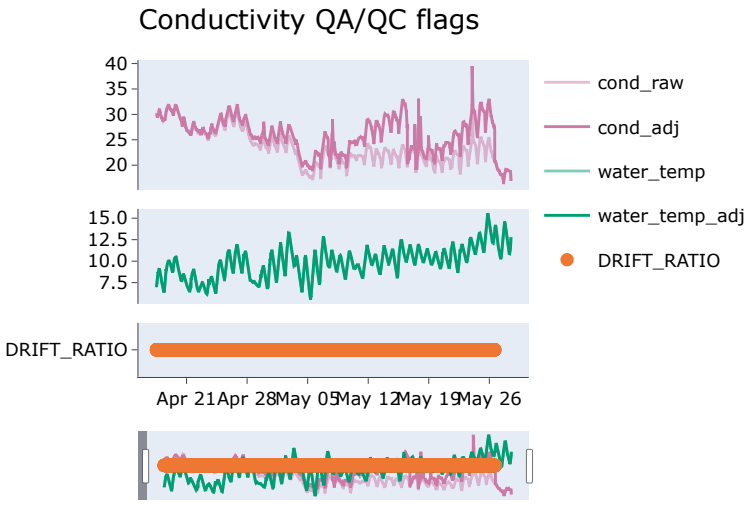
After any manual adjustments, always re-plot to verify:

```
plot_qaqc_timeseries(
  cond_data      = cond_qc,
  select_station = "TUMT_WE_40"
)
```

Step 7: Final Review

Re-plot the data after all corrections to verify the final result.

```
plot_qaqc_timeseries(
  cond_data      = cond_qc,
  select_station = "TUMT_WE_40"
)
```


Conductivity QA/QC flags
40
35
30
25
20
15.0
12.5
10.0
7.5
DRIFT_RATIO
Apr 21
Apr 28
May 05
May 12
May 19
May 26
cond_raw
cond_adj
water_temp
water_temp_adj
DRIFT_RATIO

Step 8: Export Clean Data

As of this writing, `write_clean_data()` does not support the "conductivity" metric. It supports "dissolve-doxygen", "barometric", "waterlevel", "watertemp", and "airtemp". This is a known gap in the package. For now, you can export the data manually:

```
export_cols <- c("site_station_code", "timestamp",
                 "conduct_uScm", "conduct_uScm_adj",
                 "spc_uScm_adj",
                 "watertemp_C", "watertemp_C_adj",
                 grep("^(flag_|edit_)", names(cond_qc), value = TRUE))
export_cols <- intersect(export_cols, names(cond_qc))

write.csv(
  cond_qc[, export_cols],
  file = "TUMT_WE_40_COND_v1.0.csv",
  row.names = FALSE
)
```

Recap

Step	Function	Result
1	`bind_hobo_files()` on COND	`cond_data` (raw conductivity)
2	`conductivity_qaqc()`	`cond_qc` (auto QAQC'd)
3	`plot_qaqc_timeseries()`	(visual review)
4	`wq_drift_correction()`	`cond_qc` (drift corrected)
5	`conductivity_temp_compensation()`	`cond_qc` (adds SPC)
6	Manual adjustments	`cond_qc` (updated)
7	`plot_qaqc_timeseries()`	(final review)
8	Export	CSV file

Key columns:

- `conduct_uScm` — raw electrical conductivity (never modified)
- `conduct_uScm_adj` — adjusted EC (QAQC + drift corrections applied)
- `spc_uScm_adj` — specific conductance at 25 °C (temp-compensated)
- `watertemp_C` — raw water temperature (never modified)
- `watertemp_C_adj` — adjusted water temperature
- `flag_co_*` — "something looks off — review" (data not changed)
- `edit_cond_drift` — TRUE where drift correction was applied

For analysis, use `spc_uScm_adj` — it's the fully corrected, temperature-compensated specific conductance. Compare with `conduct_uScm` to see what the processing changed.